

Apex Quant — Technical Documentation

Interactive Quantitative Trading Visualizer & Strategy Engine

This document provides a comprehensive technical overview of the **Apex Quant** platform located at **D:/projects/Trading**. It covers the trading strategy engine, real-time charting pipeline, API models, and setup guides.

1. Executive Summary

Apex Quant is an algorithmic trading visualizer and technical indicator platform. It calculates real-time pricing feeds using WebSockets, runs a technical indicator engine (EMA Ribbon + RSI Pullback), and plots live candlestick metrics, entry flags, stop-loss lines, and performance dashboards.

2. Trading Strategy Engine

The core trading algorithm implements the **EMA Ribbon + RSI Pullback** strategy.

A. Trend highway (EMA Ribbon)

The engine maintains three Exponential Moving Averages (EMA):

1. **EMA 9 (Blue)**: Represents short-term momentum.
2. **EMA 21 (Amber)**: Represents intermediate trend direction.
3. **EMA 55 (Pink)**: Acts as the structural support or resistance level.

- **Bullish Alignment**: $EMA\ 9 > EMA\ 21 > EMA\ 55$ (Uptrend)
- **Bearish Alignment**: $EMA\ 9 < EMA\ 21 < EMA\ 55$ (Downtrend)

B. Entry Rules & Execution

- **LONG Setup**:

- Trend is Bullish ($9 > 21 > 55$).
- Price pulls back to touch or approach the 21/55 EMA ribbon lines.
- RSI (14) drops below **40** (Momentum oversold).

- **SHORT Setup**:

- Trend is Bearish ($9 < 21 < 55$).
- Price bounces back to touch or approach the 21/55 EMA ribbon lines.
- RSI (14) rises above **60** (Momentum overbought).

- **Risk Management**: Enforces a strict **1:2.5 Risk-to-Reward Ratio**. The Stop Loss is placed slightly past the EMA 55 ribbon line.

Trend Highway Alignment (9, 21, 55 EMAs) ■ ■■■■ Bullish (9 > 21 > 55) ■■■ RSI < 40? ■■■
LONG Trade Entry (1:2.5 R:R) ■ ■■■■ Bearish (9 < 21 < 55) ■■■ RSI > 60? ■■■ SHORT Trade
Entry (1:2.5 R:R)

3. Technology Stack

- **Frontend:** Next.js (React 19), Zustand (State Management), TradingView Lightweight Charts (High-performance Canvas Rendering).
- **Backend:** Express.js, Node.js, WebSockets (streaming price feeds).
- **Database:** PostgreSQL (Prisma ORM for tracking transactions and logs).

4. API & WebSocket Architecture

WebSocket Feeds (Backend to Frontend)

The Express backend streams match feeds on interval:

```
{ "symbol": "BTCUSD", "price": 95243.20, "timestamp": 1719875420000, "indicators": {  
  "ema9": 95120.50, "ema21": 95080.10, "ema55": 94910.00, "rsi": 38.5 }, "signal": "LONG" }
```

Express REST Endpoints

- **GET /api/history:** Retrieves database transaction objects of past trades.
- **POST /api/settings:** Updates active strategy thresholds (e.g. changing RSI bands).
- **GET /api/status:** Health metrics of pricing feed connections.

5. Configuration & Run Guide

Create a **.env** file inside the **backend/** folder:

```
PORT=4000 DATABASE_URL=postgresql://user:password@localhost:5432/trading_db MOCK_FEED=true
```

Run Backend

1. Go to backend:

```
cd D:/projects/Trading/backend
```

2. Install and launch:

```
npm install npx prisma db push npm run dev
```

Run Frontend

1. Go to frontend:

```
cd D:/projects/Trading/frontend
```

2. Install and launch:

```
npm install npm run dev
```

Open <http://localhost:3000> to view the client.